

PRACTICAL 5

The screenshot shows a SQL IDE interface with a menu bar (File, Edit, View, Query, Database, Server, Tools, Scripting, Help) and a toolbar. The left sidebar contains a 'SCHEMAS' panel with a search filter and a list of objects: practical1, practical2, practical4, and skill3. The main editor displays SQL code for creating a database and two tables. The 'Output' panel at the bottom shows a log of executed actions with their timestamps, messages, and durations.

SQL Code:

```
1 CREATE DATABASE library_db;
2
3 USE library_db;
4
5 CREATE TABLE IF NOT EXISTS members (
6     id INT AUTO_INCREMENT PRIMARY KEY,
7     username VARCHAR(100) UNIQUE NOT NULL,
8     password_hash VARCHAR(255) NOT NULL,
9     role VARCHAR(30) NOT NULL DEFAULT 'member'
10 );
11
12 CREATE TABLE IF NOT EXISTS books (
13     id INT AUTO_INCREMENT PRIMARY KEY,
14     title VARCHAR(250) NOT NULL,
15     author VARCHAR(200) NOT NULL,
16     isbn VARCHAR(13) UNIQUE NOT NULL,
```

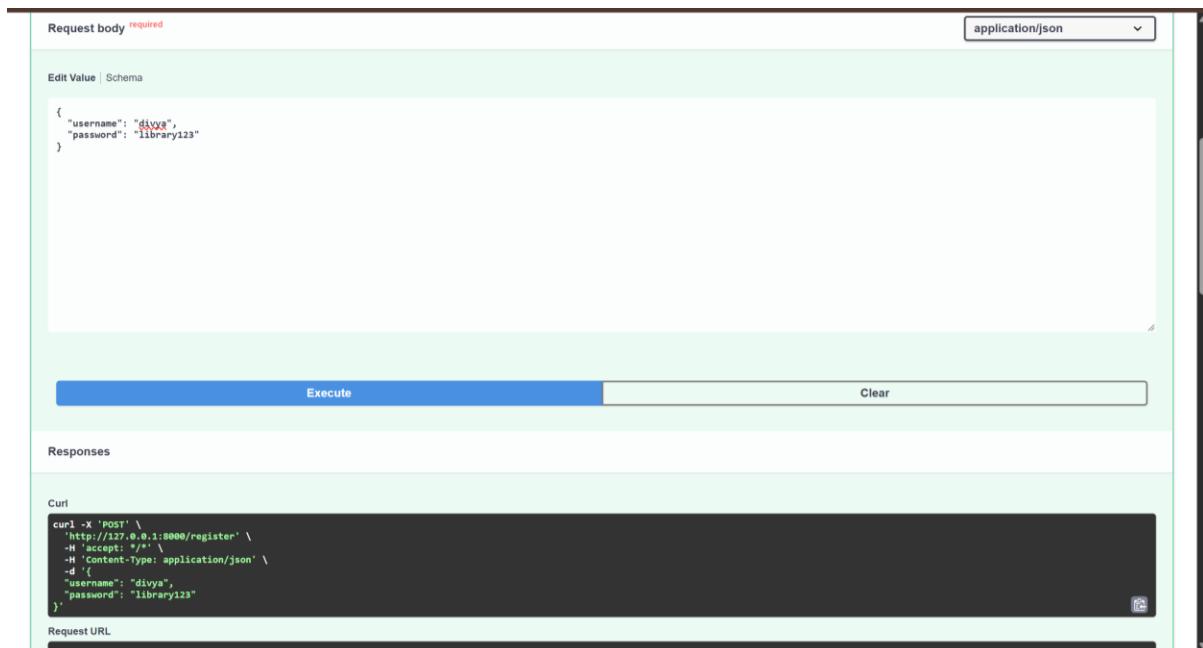
Output Log:

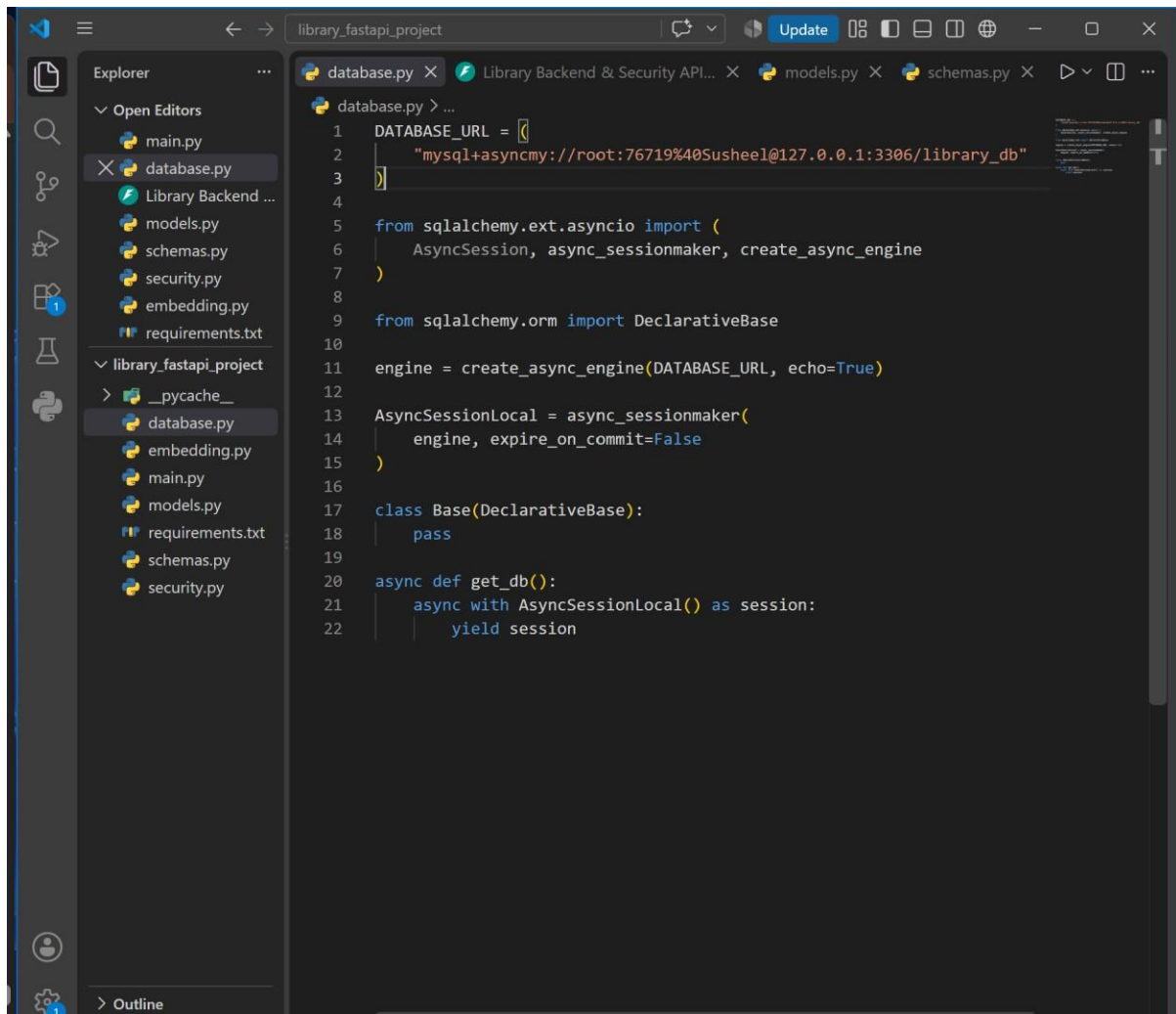
#	Time	Action	Message	Duration / Fetch
✓ 17	23:18:20	UPDATE Books SET embedding = JSON_ARRA...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.015 sec
✓ 18	23:18:33	UPDATE Books SET embedding = JSON_ARRA...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
✓ 19	23:18:42	UPDATE Books SET embedding = JSON_ARRA...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.016 sec
✓ 20	23:18:55	UPDATE Books SET embedding = JSON_ARRA...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.031 sec
✓ 21	23:19:06	SELECT book_id, title, embedding FROM ...	10 row(s) returned	0.000 sec / 0.000 sec
✓ 22	23:20:07	SELECT book_id, title, author, genre, ...	3 row(s) returned	0.015 sec / 0.000 sec
✓ 23	23:21:08	SELECT book_id, title, author, genre, ...	3 row(s) returned	0.000 sec / 0.000 sec
✓ 24	22:51:53	CREATE DATABASE library_db	1 row(s) affected	0.078 sec
✓ 25	22:52:11	USE library_db	0 row(s) affected	0.016 sec
✓ 26	22:52:26	CREATE TABLE IF NOT EXISTS members (id ...	0 row(s) affected	0.078 sec
✓ 27	22:53:33	CREATE TABLE IF NOT EXISTS books (id IN...	0 row(s) affected	0.031 sec

VS Code editor showing a Python project setup for a FastAPI application. The Explorer sidebar shows files like `main.py`, `database.py`, `models.py`, `schemas.py`, `security.py`, `embedding.py`, `requirements.txt`, and `__pycache__`. The `database.py` file is open, showing the database URL, SQLAlchemy engine setup, and a `Base` class.

```
1 DATABASE_URL = (  
2     "mysql+asyncmy://root:76719%40Susheel@127.0.0.1:3306/library_db"  
3 )  
4  
5 from sqlalchemy.ext.asyncio import (  
6     AsyncSession, async_sessionmaker, create_async_engine  
7 )  
8  
9 from sqlalchemy.orm import DeclarativeBase  
10  
11 engine = create_async_engine(DATABASE_URL, echo=True)  
12  
13 AsyncSessionLocal = async_sessionmaker(  
14     engine, expire_on_commit=False  
15 )  
16  
17 class Base(DeclarativeBase):  
18     pass  
19
```

The Terminal shows the command `python -m uvicorn main:app --reload` and its output, including database connection logs and "Application startup complete."

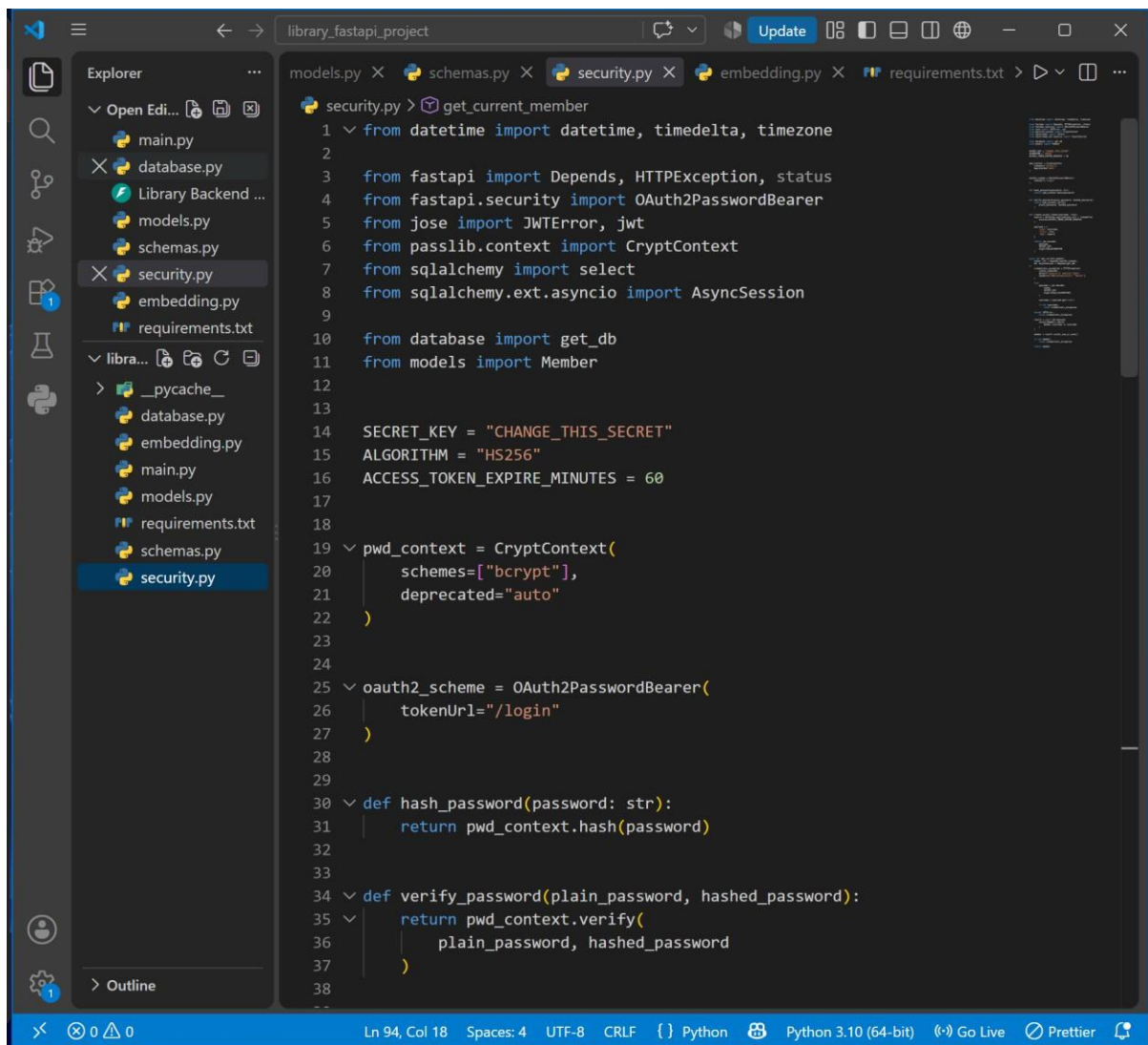


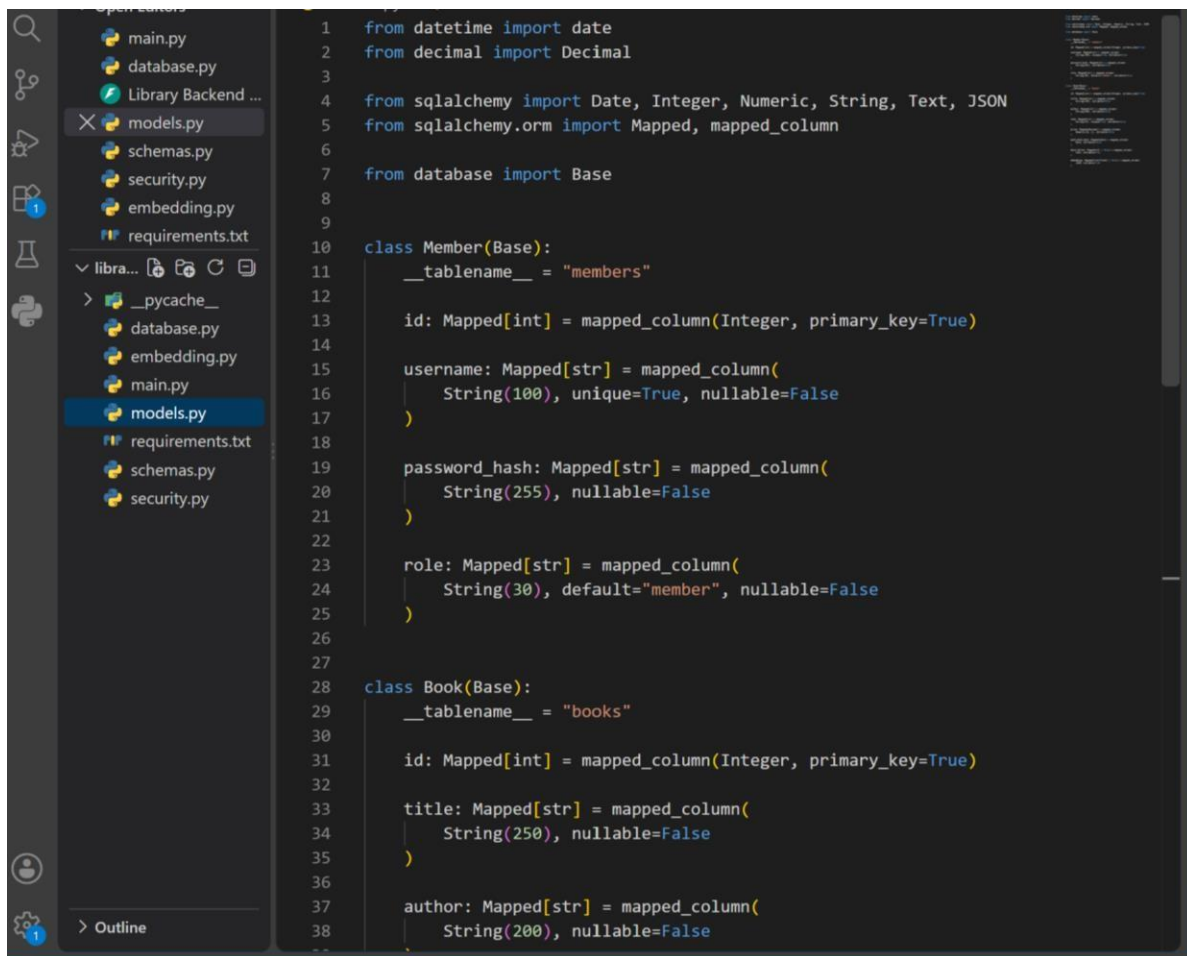


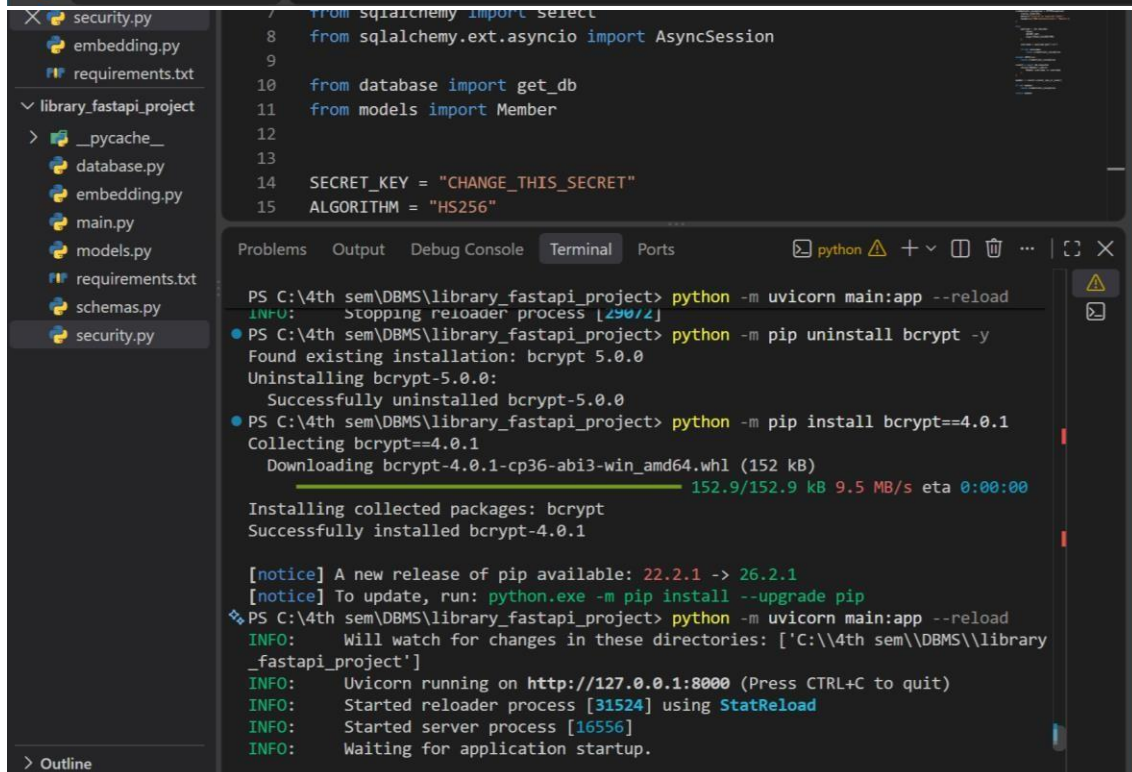
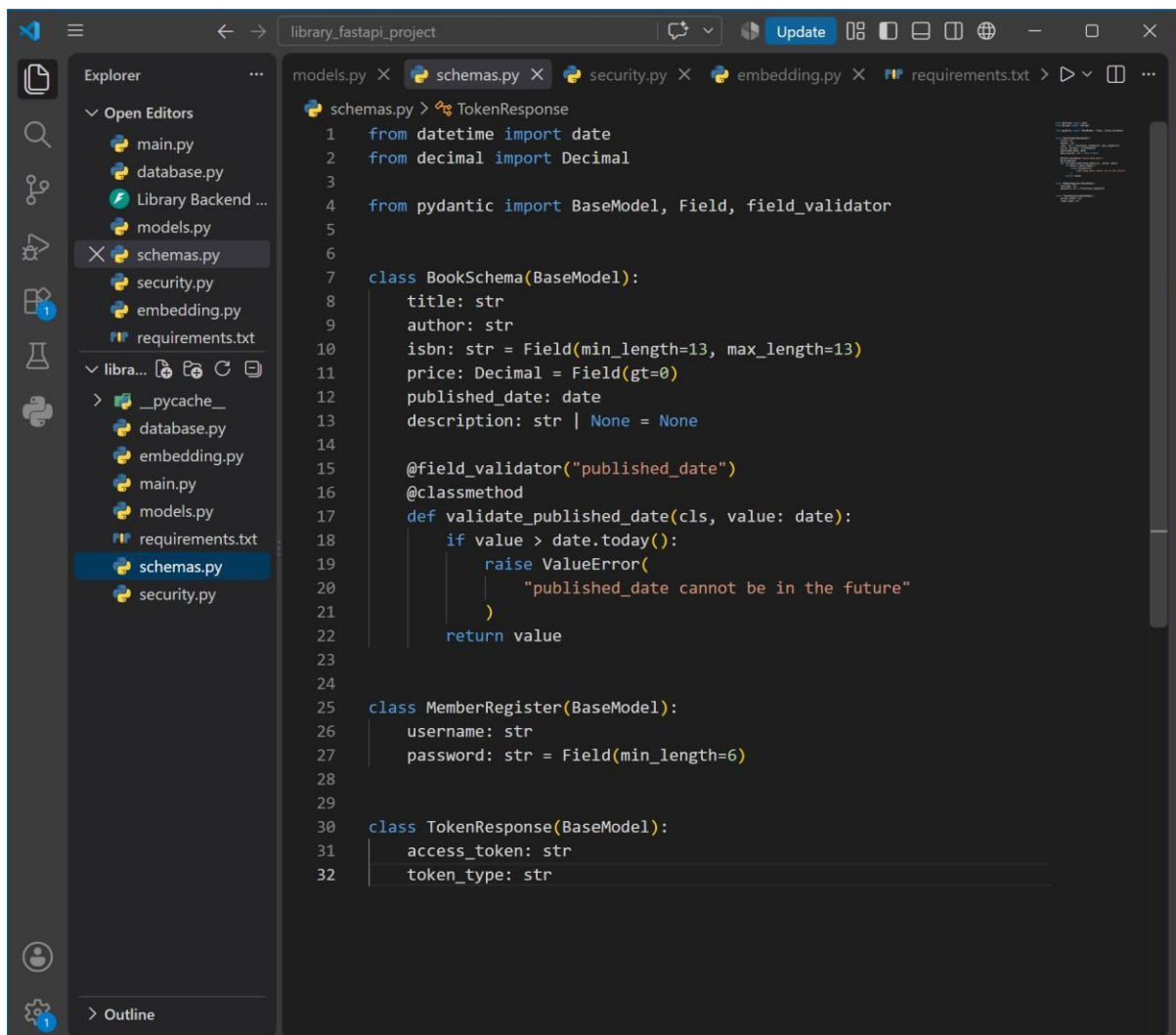
The image shows a Visual Studio Code editor window with the following components:

- Explorer Panel (Left):** Displays the project structure for 'library_fastapi_project'. The 'embedding.py' file is selected and highlighted in blue.
- Editor Panel (Center):** Shows the code for 'embedding.py'. The function 'text_to_vector' is defined, which takes a string 'text' and returns a list of floats. The function uses 'hashlib.sha256' to generate a digest, converts it to an integer, and then calculates a magnitude to normalize the values.
- Terminal/Status Bar (Bottom):** Shows the current line and column (Ln 28, Col 43), the number of spaces (4), the encoding (UTF-8), the line ending (CRLF), the language (Python), the Python version (Python 3.10 (64-bit)), and the Prettier extension.

```
1 import hashlib
2 import math
3
4 VECTOR_DIM = 384
5
6
7 def text_to_vector(text: str) -> list[float]:
8     values = []
9
10    for i in range(VECTOR_DIM):
11        digest = hashlib.sha256(
12            f"{text.lower()}:{i}".encode()
13        ).digest()
14
15        number = int.from_bytes(
16            digest[:4],
17            byteorder="big"
18        )
19
20        values.append(
21            (number / 2**32) * 2 - 1
22        )
23
24    magnitude = math.sqrt(
25        sum(x * x for x in values)
26    )
27
28    return [x / magnitude for x in values]
```







Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "divya",
    "password": "library123"
  }'
```

Request URL

http://127.0.0.1:8000/register

Server response

Code	Details
------	---------

200

Response body

```
{
  "message": "Member registered successfully",
  "username": "divya",
  "role": "member"
}
```

Response headers

```
content-length: 79
content-type: application/json
date: Sun, 20 Sep 2026 18:01:42 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links
Media type		
application/json		
Controls Accept header.		